



University of Vienna
Faculty of Computer Science
Praktikum Informatik 2
Summer Semester 2026

A Domain-Specific Language for Customizable Fact Comparison Logic in FactCheck

Khalifa Nikzad
a12437813@unet.univie.ac.at

Supervisor: Daniel Berger

Vienna, Austria
July 1, 2026

Contents

1	Introduction	4
1.1	Report Structure	4
2	Problem Statement and Motivation	5
2.1	Fact Comparison Challenges	5
2.2	Limitations of Traditional Comparison	5
2.3	Motivation for a Domain-Specific Language	5
3	Project Objectives and Requirements	6
3.1	Project Goal	6
3.2	Functional Requirements	6
3.3	Non-functional Requirements	6
3.4	Expected Results	7
4	Proposed Solution	7
4.1	Design Overview	7
4.2	Key Design Decisions	7
4.3	Alternative Design Approaches	7
4.4	Formula Processing Workflow	8
5	System Architecture	8
5.1	Frontend	9
5.2	Backend	9
5.3	Database	9
5.4	Deployment	10
6	Domain-Specific Language	10
6.1	Language Overview	10
6.2	Formula Syntax	10
6.3	Predefined Comparison Functions	10
6.4	Example Formula	11
6.5	Abstract Syntax Tree	11
7	Formula Validation and Execution	11
7.1	Formula Creation	12
7.2	Validation Pipeline	12
7.2.1	Syntax Validation	12
7.2.2	Semantic Validation	13
7.2.3	Structural Validation	13
7.2.4	Safety Validation	13
7.3	Formula Storage	13
7.4	Formula Execution	13
8	System Implementation	14
8.1	Backend Implementation	14
8.2	Frontend Implementation	14
8.3	Database Design	15
8.4	Function Synchronization	16
8.5	Containerization	16

9	Evaluation	16
9.1	Functional Evaluation	16
9.2	Example Comparison	16
9.3	Discussion	17
10	Challenges, Limitations and Future Work	17
10.1	Challenges	18
10.2	Limitations	18
10.3	Future Work	18
11	Conclusion	19

Abstract

Fact comparison is an important task in data integration and fact-checking systems, where information from multiple sources must be compared to determine whether different records describe the same real-world entity. Traditional comparison approaches often rely on fixed comparison logic implemented directly in application code, making them difficult to customize, maintain, and extend.

This project presents a domain-specific language (DSL) for defining customizable fact comparison logic within the FactCheck framework. The proposed DSL allows users to create reusable comparison formulas by combining predefined comparison functions with logical, arithmetic, comparison, and conditional operators without modifying the application source code. To ensure safe and reliable execution, formulas are parsed into an Abstract Syntax Tree (AST), validated through a multi-stage validation pipeline, and executed by a controlled execution engine.

The implemented prototype consists of a React frontend, a FastAPI backend, a MongoDB database, and a Docker-based deployment environment. The framework also supports formula versioning, automatic synchronization of predefined comparison functions, execution history, and explainable comparison results with confidence scores.

The evaluation demonstrates that the developed framework successfully supports the complete lifecycle of customizable comparison formulas, including creation, validation, storage, and execution. The results show that the proposed approach provides a flexible, maintainable, and extensible solution for defining comparison logic while ensuring safe execution and transparent comparison results.

1 Introduction

Many websites publish information about the same real-world entities, such as movies, books, products, or people. Although these sources often describe the same fact, the published values are not always identical. Differences in formatting, naming conventions, abbreviations, or small numerical variations make automatic comparison difficult. As a result, a simple equality check is often not enough to determine whether two values represent the same fact.

The FactCheck framework [1] was developed to support the comparison and analysis of information collected from different data providers. It covers the complete comparison process, including data acquisition, fact comparison, result presentation, and further analysis. One important challenge in this process is defining how two facts should be compared. Different comparison scenarios require different comparison strategies, and a single comparison function cannot provide accurate results for every case.

This project addresses this challenge by developing a domain-specific language (DSL) that enables customizable fact comparison. Instead of implementing comparison logic directly in source code, users can define comparison formulas by combining predefined comparison functions with logical, arithmetic, comparison, and conditional operators. This approach allows domain experts to create and maintain comparison rules without modifying the implementation of the system.

To ensure reliable execution, every formula follows a controlled processing pipeline. User-defined formulas are parsed into an Abstract Syntax Tree (AST) [2], validated through multiple validation stages, stored in a database, and executed by a dedicated execution engine. During execution, the system produces not only the comparison result but also an explanation and a confidence value, making the decision process easier to understand.

The final prototype consists of a FastAPI backend, a React web application, a MongoDB database, and Docker support for deployment. Together, these components provide a complete environment for creating, validating, storing, and executing customizable comparison formulas through a user-friendly web interface.

The main contribution of this project is the design and implementation of a domain-specific language together with an AST-based validation and execution framework that enables customizable, safe, and explainable fact comparison without requiring modifications to the application source code.

1.1 Report Structure

The remainder of this report is organized as follows:

- **Chapter 2** presents the problem statement and explains the motivation behind customizable fact comparison.
- **Chapter 3** describes the project objectives and requirements.
- **Chapter 4** introduces the proposed solution and its main design ideas.
- **Chapter 5** presents the overall system architecture and explains the interaction between the different components.
- **Chapter 6** describes the design of the domain-specific language (DSL), including the supported formula syntax and the complete formula lifecycle.
- **Chapter 7** explains the formula validation and execution process.
- **Chapter 8** presents the implementation of the backend, frontend, database, and deployment environment.
- **Chapter 9** evaluates the implemented system using practical examples and discusses the obtained results.

- **Chapter 10** discusses the main challenges, limitations, and possible future improvements.
- **Chapter 11** concludes the report.

2 Problem Statement and Motivation

2.1 Fact Comparison Challenges

Many organizations and online platforms publish information about the same real-world entities. For example, multiple websites may provide information about the same movie, product, book, or person. Although these sources describe the same fact, the published values are often represented differently. Titles may use different formatting, dates may appear in different formats, and numerical values may vary slightly. These differences make automatic fact comparison difficult.

A simple equality check can only determine whether two values are exactly the same. In practice, however, different representations may still describe the same fact. For example, the movie titles "*RoboCop*" and "*Robocop (1987)*" refer to the same movie, even though the two strings are not identical. Similar situations also occur when comparing dates, durations, names, or lists of actors. Therefore, fact comparison requires more flexible comparison methods than simple string or value equality.

2.2 Limitations of Traditional Comparison

Different comparison scenarios require different comparison strategies. Some values should match exactly, while others should allow small differences or use semantic comparison methods. For example, publication years may require an exact comparison, while movie durations can still represent the same release if they differ by only a few minutes. Likewise, titles often need normalization before they can be compared correctly.

Implementing these comparison rules directly in application code has several disadvantages. Every new comparison scenario requires source code modifications, testing, and deployment. As the number of comparison rules grows, maintaining the implementation becomes more difficult and less flexible. In addition, domain experts who understand the comparison logic usually do not have direct access to modify the application code.

2.3 Motivation for a Domain-Specific Language

The FactCheck project [1] addresses these challenges by providing a framework for comparing information obtained from different data sources. As part of this framework, this project introduces a domain-specific language (DSL) [3] for customizable fact comparison. Instead of writing comparison logic directly in Python, users can define comparison formulas by combining predefined comparison functions with logical, arithmetic, comparison, and conditional operators.

This approach separates comparison logic from the application implementation. Developers are responsible for providing safe and reusable comparison functions, while domain experts can create and update comparison formulas without modifying the source code. As a result, the system becomes easier to maintain, more flexible, and better suited for different comparison scenarios.

Since user-defined formulas influence the comparison process, they must be validated before execution. The system therefore performs syntax, semantic, structural, and safety validation before a formula is stored. During execution, additional validation ensures that the required input data is available and correctly formatted. This controlled approach allows users to customize comparison logic while maintaining reliable and secure execution.

3 Project Objectives and Requirements

This chapter presents the main objectives and requirements of the project. They are based on the agreed project definition [1] and describe the expected functionality of the system before the implementation phase. The purpose of these requirements is to ensure that the developed system provides a flexible, safe, and user-friendly environment for creating and executing customizable comparison logic.

3.1 Project Goal

The main goal of this project is to develop a domain-specific language (DSL) that enables domain experts to define customizable comparison logic through formulas. Instead of implementing comparison rules directly in the application code, users should be able to combine predefined comparison functions using logical, arithmetic, comparison, and conditional operators.

The created formulas should be validated before they are stored, executed safely on demand, and produce transparent comparison results. During formula creation, the system should provide clear validation feedback, while both validation and execution activities should be recorded to improve traceability and support future analysis.

3.2 Functional Requirements

The system should provide the following functionality:

1. Create comparison formulas using predefined comparison functions and supported operators.
2. Validate formulas before storage.
3. Store formulas with version management.
4. Execute formulas using user-provided input data.
5. Manage predefined comparison functions.
6. Record validation and execution history.
7. Provide a web interface for creating, managing, and executing formulas.

3.3 Non-functional Requirements

In addition to the required functionality, the system should satisfy several quality requirements.

- Execute formulas safely without evaluating arbitrary Python code.
- Provide meaningful validation messages to help users identify and correct invalid formulas.
- Generate explainable comparison results instead of returning only a boolean value.
- Support reusable and extensible predefined comparison functions.
- Maintain formula versions and execution history to ensure transparency and traceability.
- Provide a responsive and user-friendly web interface.
- Support containerized deployment using Docker.

3.4 Expected Results

According to the project definition, the expected outcome is a fully functional prototype that enables users to define, validate, store, and execute customizable comparison formulas through a web application. The system should provide a secure execution environment, maintain version history, record validation and execution information, and generate explainable comparison results. These objectives guided both the design and implementation of the final prototype presented in this report.

4 Proposed Solution

4.1 Design Overview

To address the limitations of traditional comparison approaches, this project proposes a domain-specific language (DSL) [3] for customizable fact comparison. Instead of implementing comparison logic directly in the application code, users define comparison rules by combining predefined comparison functions with logical, arithmetic, comparison, and conditional operators.

The proposed solution separates the responsibilities of developers and domain experts. Developers implement reusable comparison functions, while domain experts create comparison formulas that describe how facts should be compared. This separation allows new comparison scenarios to be supported without modifying the application source code.

Rather than executing user-defined formulas directly, the system processes every formula through a controlled workflow. Each formula is parsed, validated, stored, and executed using dedicated system components. This approach improves safety, maintainability, and flexibility while ensuring that only valid comparison logic can be executed.

4.2 Key Design Decisions

Several important design decisions guided the development of the system.

First, a domain-specific language was selected instead of allowing users to write Python code. Restricting users to a predefined language makes it possible to validate formulas before execution and prevents arbitrary code execution.

Second, comparison logic is built from predefined comparison functions. Each function performs a specific comparison task, such as comparing titles, dates, numerical ranges, or object properties. Because these functions are reusable, they can be combined in different ways to support many comparison scenarios.

Another important decision was to validate formulas before storing them in the database. Invalid formulas are rejected during the creation process, reducing runtime errors and improving the overall reliability of the system.

Finally, the project uses formula versioning and execution history to improve traceability. Every valid formula version is stored together with its metadata, while each execution is recorded for later review and analysis.

4.3 Alternative Design Approaches

Several alternative approaches were considered before selecting the final solution.

One possible approach is to implement fact comparison using a traditional rule-based system. In such a system, comparison rules are predefined by developers and users can only configure or select existing rules. Although this approach is easy to implement and validate, it provides limited flexibility because supporting new comparison scenarios requires modifications to the application. Since the objective of this project is to enable domain experts to define customizable comparison logic, a purely rule-based solution was considered insufficient.

Another alternative is to evaluate user-defined formulas directly as text using Python's dynamic evaluation mechanisms, such as `eval()`. While this approach simplifies implementation, it executes arbitrary code and therefore introduces significant security risks. Furthermore, formulas cannot easily be validated before execution, making it difficult to guarantee correctness, safety, and predictable behaviour.

The implemented solution adopts an Abstract Syntax Tree (AST)-based approach combined with a registry of predefined comparison functions. User-defined formulas are first parsed into an AST and validated through syntax, semantic, structural, and safety validation before they are stored. During execution, only registered comparison functions and supported operators can be evaluated by the execution engine.

Compared with the alternative approaches, the selected solution provides a balance between flexibility and safety. Restricting the language to predefined comparison functions intentionally reduces its expressiveness compared with a general-purpose programming language. However, this design enables comprehensive validation before execution, prevents arbitrary code execution, and improves the safety, predictability, maintainability, and traceability of the framework.

4.4 Formula Processing Workflow

The complete lifecycle of a comparison formula consists of several processing stages. A user first creates a formula using the web interface. The formula is then parsed into an Abstract Syntax Tree (AST) and passes multiple validation stages, including syntax, semantic, structural, and safety validation. If the formula is valid, it is stored in the database together with its metadata and version information.

When the user executes a formula, the system first validates the provided input data. The execution engine then evaluates the stored AST by invoking the required predefined comparison functions. Finally, the system returns the comparison result together with an explanation and a confidence score, allowing users to understand how the final decision was produced.

Figure 1 illustrates the overall formula processing workflow, from formula creation and validation to execution and result generation.

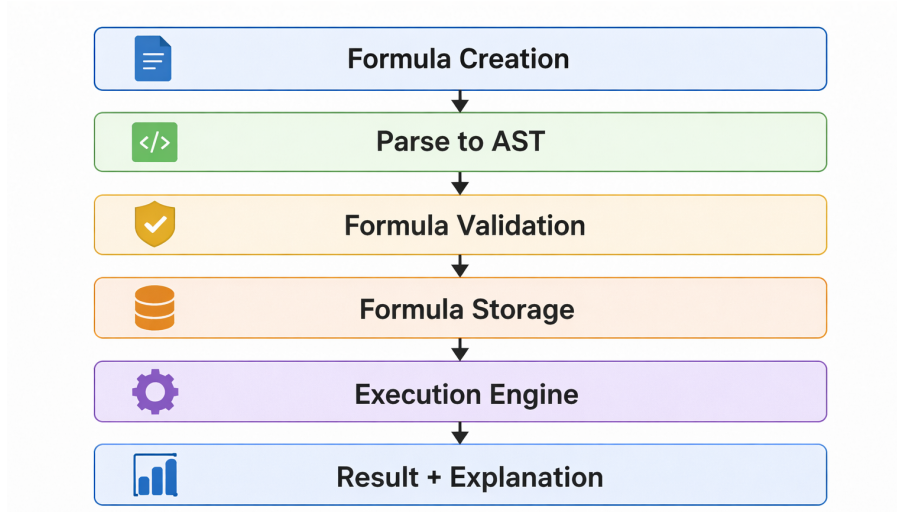


Figure 1: Overall formula processing workflow.

5 System Architecture

The previous chapter introduced the overall solution and the conceptual workflow of formula processing. This chapter presents the architecture of the implemented prototype and explains how its

main software components interact to support formula creation, validation, storage, and execution.

The system follows a client-server architecture consisting of a React frontend [4], a FastAPI backend [5], and a MongoDB database [6]. Each component has a clearly defined responsibility, resulting in a modular and maintainable system. Figure 2 illustrates the overall architecture of the implemented prototype.

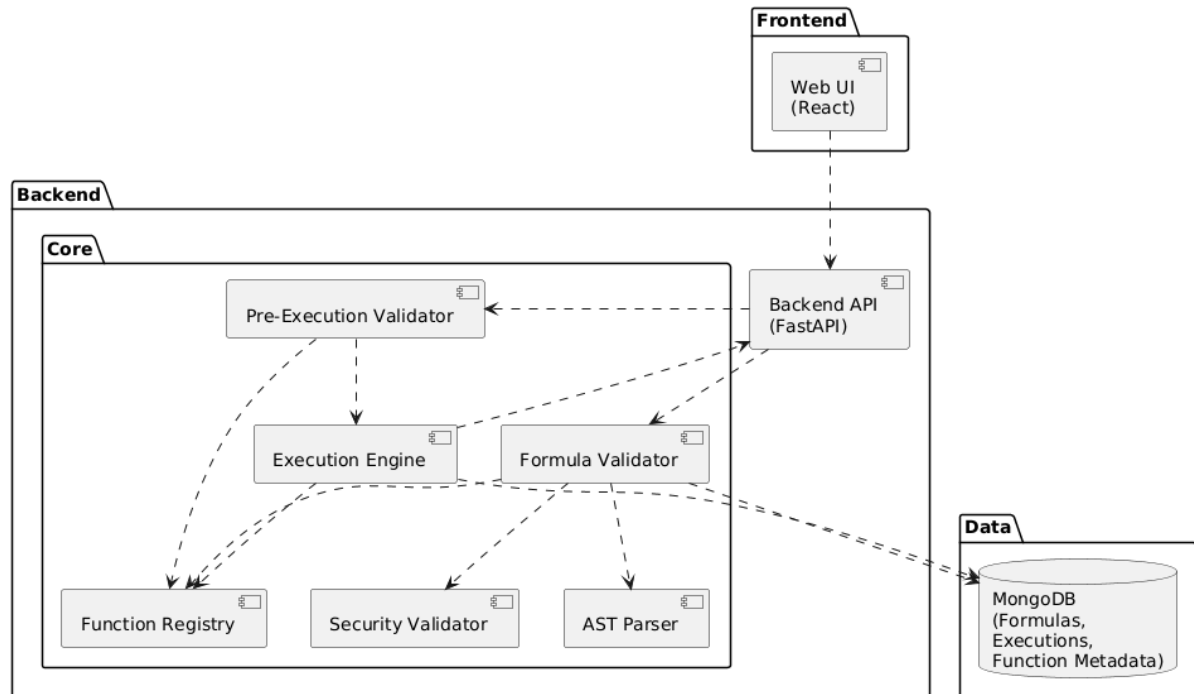


Figure 2: Logical architecture of the implemented FactCheck comparison system.

5.1 Frontend

The frontend was developed using React [4] and serves as the primary interface for interacting with the system. Through the web application, users can create comparison formulas, validate and execute stored formulas, browse the available predefined comparison functions, and review formula and execution history. Communication with the backend is performed through REST APIs.

5.2 Backend

The backend was implemented using FastAPI [5] and contains the core business logic of the system. It provides REST endpoints for formula management, validation, execution, function discovery, dashboard statistics, and history retrieval. In addition, it coordinates the internal components responsible for formula processing and manages communication with the database.

5.3 Database

MongoDB [6] is used as the persistence layer of the system. It stores formulas, formula versions, predefined function metadata, and execution history. This organization allows the system to maintain version information, support traceability, and efficiently retrieve the data required during formula validation and execution.

5.4 Deployment

To simplify deployment and improve portability, the application is containerized using Docker [7]. Docker Compose orchestrates the frontend, backend, and MongoDB containers, providing a consistent execution environment for both development and demonstration.

6 Domain-Specific Language

This chapter introduces the domain-specific language (DSL) [3] developed for defining customizable comparison logic. Instead of implementing comparison rules directly in Python, the DSL enables users to express comparison logic by combining predefined comparison functions with logical, arithmetic, comparison, and conditional operators. This approach makes comparison formulas easier to understand, reuse, and maintain.

6.1 Language Overview

The DSL was designed for domain experts who need to define comparison rules without modifying the application source code. Rather than writing general-purpose programs, users create formulas using a small and controlled language that supports only the operations required for fact comparison.

Every formula follows the processing workflow presented in the previous chapter. After a user creates a formula, it is parsed, validated, stored, and later executed by the comparison engine. Since users can only use predefined comparison functions and supported operators, the system can validate formulas before execution and prevent unsafe operations.

6.2 Formula Syntax

A comparison formula consists of predefined comparison functions, variables, constants, and supported operators. Functions are written using the following general syntax:

```
functionName(parameter1, parameter2, ...)
```

The DSL supports the following language elements:

- Logical operators: AND, OR, NOT
- Comparison operators: ==, !=, <, >, <=, >=
- Arithmetic operators: +, -, *, /
- Conditional expressions using the IF() function
- Calls to predefined comparison functions

These language elements allow users to create both simple and complex comparison formulas while keeping the language easy to understand.

6.3 Predefined Comparison Functions

The DSL is built on a collection of predefined comparison functions. Each function performs a specific comparison task and can be reused in different formulas. Developers can extend the system by implementing additional predefined comparison functions, while users can immediately use these functions when creating new comparison formulas.

Some of the predefined function categories implemented in the system are listed below.

- Equality comparison (e.g., `totallyEqual()`)
- Text comparison (e.g., `normalizedTitleMatch()`)
- Range comparison (e.g., `rangeAbsolute()`, `rangePercentage()`)
- Date comparison (e.g., `sameYear()`)
- Collection comparison (e.g., `valueInList()`)

6.4 Example Formula

The following example demonstrates the comparison formula used throughout this report.

Formula Name

Movie Metadata Consistency Check

Description

Checks whether two movie records likely describe the same release by comparing title similarity, publication year, and duration tolerance.

Formula Expression

```
normalizedTitleMatch(name1, name2)
AND
sameYear(datePublished1, datePublished2)
AND
rangeAbsolute(duration1, duration2, 20)
```

This formula first compares the normalized movie titles. It then checks whether both publication dates refer to the same year and finally verifies that the duration difference between the two movies does not exceed twenty minutes. Since the three conditions are connected using the logical AND operator, all of them must evaluate to true for the comparison to be considered successful.

6.5 Abstract Syntax Tree

User-defined formulas are never executed directly. Instead, every formula is parsed into an Abstract Syntax Tree (AST) [2]. The AST provides a structured representation of the formula that separates operators, function calls, variables, and constants into individual nodes. This representation allows the system to validate the formula before it is stored and later evaluate it safely during execution.

Using an AST separates parsing from execution and supports syntax validation, semantic validation, structural validation, and safety validation. After successful validation, the AST is stored together with the corresponding formula version and is later evaluated by the execution engine.

Figure 3 shows the AST generated from the Movie Metadata Consistency Check formula presented above. The root node represents the logical AND operator, while each child node corresponds to one predefined comparison function. The leaf nodes represent the input variables and constant values used by the comparison functions.

7 Formula Validation and Execution

This chapter describes how the system processes a comparison formula after it is submitted by a user. The process consists of two main phases. During formula creation, the system validates the formula and stores a validated representation in the database. During execution, the stored formula is evaluated using user-provided input data to produce an explainable comparison result.

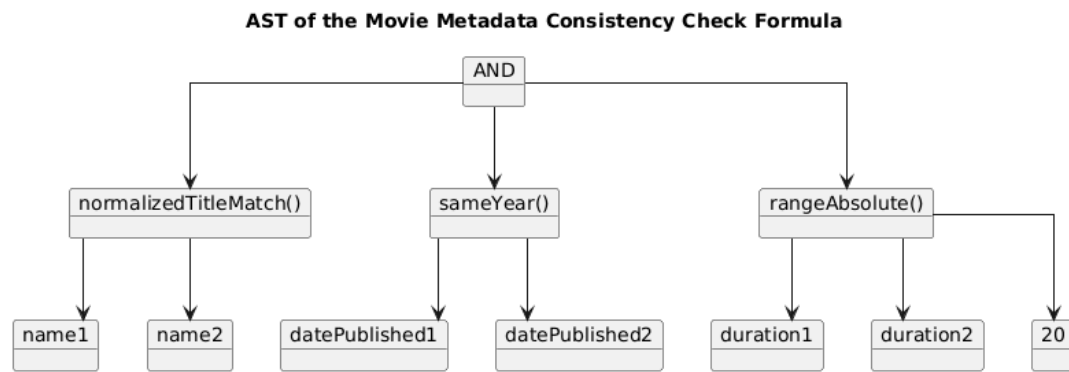


Figure 3: Abstract Syntax Tree (AST) of the example comparison formula.

Separating validation from execution improves reliability because only validated formulas can be executed. It also allows formulas to be reused multiple times without repeating the validation process.

7.1 Formula Creation

When a user creates a new comparison formula, the backend first parses the submitted expression into an Abstract Syntax Tree (AST). If the expression cannot be parsed because of syntax errors, the corresponding validation errors are recorded and returned to the user. Otherwise, the generated AST is used throughout the remaining validation process.

The backend then performs several validation steps to ensure that the formula is correct, safe, and executable.

The validation process includes:

- Syntax validation
- Semantic validation
- Structural validation
- Safety validation

If all validation steps succeed, the formula, together with its generated AST, metadata, version information, and validation details, is stored as a valid formula. If one or more validation steps fail, the failed creation attempt is recorded together with the validation errors, allowing users to review unsuccessful submissions and improving traceability of the formula creation process.

7.2 Validation Pipeline

Formula validation is performed in multiple stages. Each stage focuses on a different aspect of correctness.

7.2.1 Syntax Validation

The parser first checks whether the formula follows the grammar of the DSL. Invalid expressions, missing parentheses, or unsupported operators are rejected before further processing.

7.2.2 Semantic Validation

The semantic validator verifies that all referenced functions exist in the function registry and that the number of arguments matches the corresponding function definition.

7.2.3 Structural Validation

The AST is analysed to ensure that every node represents a supported expression type and that the overall structure can be evaluated correctly.

7.2.4 Safety Validation

The safety validator prevents unsupported or unsafe constructs. Only predefined comparison functions and supported operators are allowed, ensuring that arbitrary Python code cannot be executed.

7.3 Formula Storage

After successful validation, the formula is stored in the database. Each stored formula contains general information such as its name and description, while every modification creates a new version. Each version stores the formula expression, the generated AST, the referenced predefined comparison functions, and the validation information. This design allows formulas to evolve over time while preserving previous versions for traceability.

7.4 Formula Execution

To illustrate the execution process, the Movie Metadata Consistency Check formula introduced in the previous chapter is used throughout this section.

Formula Expression

```
normalizedTitleMatch(name1, name2)
AND
sameYear(datePublished1, datePublished2)
AND
rangeAbsolute(duration1, duration2, 20)
```

Users may execute this formula using either flat key-value pairs or structured records following the Schema.org Thing model [8].

Flat Key-Value Input

Listing 1: Flat input data accepted by the execution engine.

```
1 {
2   "name1": "RoboCop",
3   "name2": "Robocop (1987)",
4   "datePublished1": "1987-07-17",
5   "datePublished2": "July 17, 1987",
6   "duration1": 102,
7   "duration2": 105
8 }
```

Schema.org-style Input

Listing 2: Schema.org-style input data accepted by the execution engine.

```
1 {  
2   "thing1": {  
3     "@type": "Thing",  
4     "name": "RoboCop",  
5     "datePublished": "1987-07-17",  
6     "duration": 102  
7   },  
8   "thing2": {  
9     "@type": "Thing",  
10    "name": "Robocop (1987)",  
11    "datePublished": "July 17, 1987",  
12    "duration": 105  
13  }  
14 }
```

Both input formats are supported by the execution engine. When flat key-value pairs are provided, the variables are used directly during execution. When Schema.org-style records are supplied, the framework automatically maps object properties to the variables referenced by the comparison formula before evaluation. For example, `thing1.name` is mapped to `name1`, while `thing2.duration` is mapped to `duration2`. This preprocessing step allows the same comparison formula to be executed using either input format without modifying the formula itself.

After preprocessing, the system performs a lightweight pre-execution validation to verify that all required variables are available and that the referenced predefined comparison functions exist. The execution engine then evaluates the stored AST recursively. Variables are replaced by their corresponding values, predefined comparison functions are invoked, and logical operators combine intermediate results until the final comparison result is produced. Besides the comparison result, the execution engine also generates a detailed explanation and a confidence score, allowing users to understand how the final decision was obtained.

8 System Implementation

This chapter describes the implementation of the proposed framework. The prototype consists of a React frontend, a FastAPI backend, a MongoDB database, and Docker-based deployment. The implementation follows a modular architecture that separates user interaction, business logic, data storage, and deployment.

8.1 Backend Implementation

The backend was implemented using Python [9] and FastAPI [5]. FastAPI was selected because it provides automatic request validation, OpenAPI documentation, and high performance for RESTful web services.

The backend is organized into multiple modules that separate API endpoints, business logic, validation, execution, and database access. This modular structure improves maintainability and allows individual components to evolve independently. The backend exposes REST endpoints for formula management, validation, execution, function discovery, dashboard statistics, and history retrieval. Internally, it coordinates the parser, validation pipeline, execution engine, function registry, and database layer to process comparison formulas throughout their lifecycle.

8.2 Frontend Implementation

The frontend was developed using React [4] together with Vite. Bootstrap was used for the user interface, while Axios handles communication with the backend through REST APIs.

The web interface allows users to create comparison formulas, browse predefined comparison functions, execute stored formulas, and review validation and execution history. A CodeMirror editor provides syntax highlighting during formula creation, improving readability and the overall user experience.

8.3 Database Design

MongoDB [6] was selected as the persistence layer because the system stores flexible and evolving document structures, such as formulas, Abstract Syntax Trees (ASTs), predefined comparison function metadata, validation information, and execution history. Unlike relational databases, MongoDB stores data as documents and allows related information to be embedded within parent documents, reducing the need for joins and simplifying data retrieval.

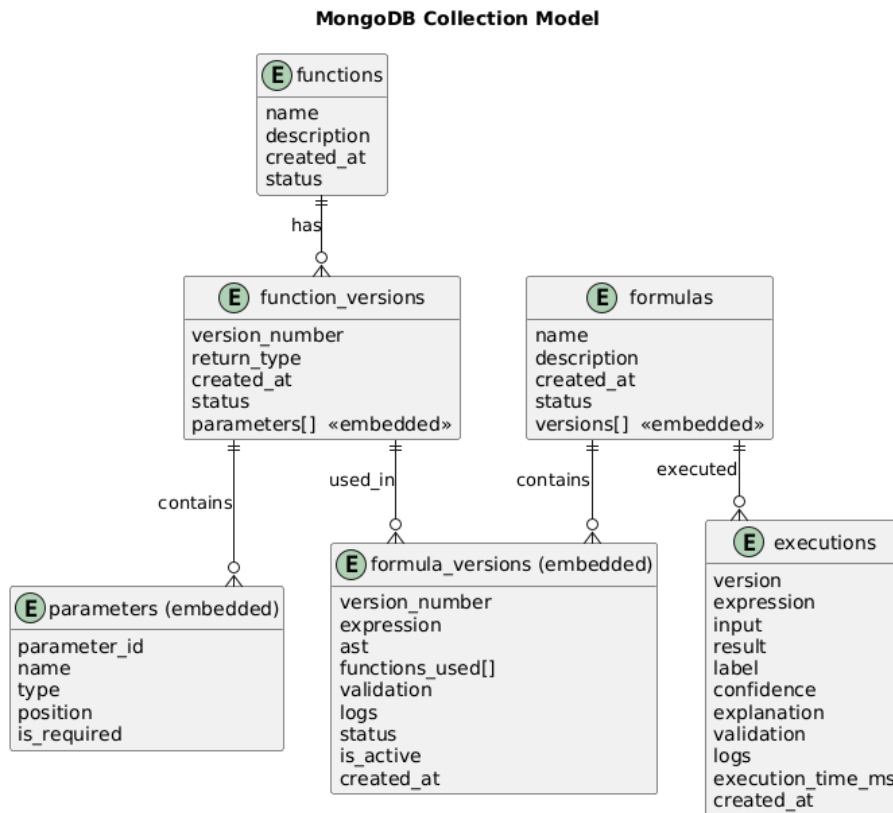


Figure 4: MongoDB collection model of the implemented system.

Figure 4 illustrates the MongoDB collection model implemented by the system. The database consists of four primary collections: *Functions*, *Function Versions*, *Formulas*, and *Executions*. Function parameters are embedded within function versions, while formula versions are embedded within formulas together with their AST, validation information, referenced comparison functions, and version metadata. Execution records are stored separately to preserve execution history and improve traceability.

This document-oriented design preserves version history while reducing data fragmentation through embedded documents. As a result, the implemented database model supports efficient retrieval of formulas and execution results while remaining flexible enough to accommodate future extensions of the comparison framework.

8.4 Function Synchronization

The framework automatically synchronizes predefined comparison functions with the database during application startup. Instead of manually registering every function, the synchronization process scans the predefined function directory, extracts metadata using Python reflection, and compares the discovered functions with those already stored in the database.

Only functions that are not yet registered are inserted into the database. This approach simplifies maintenance because developers only need to implement new predefined comparison functions. The synchronization process automatically makes them available to the DSL without requiring manual database updates.

8.5 Containerization

The application is containerized using Docker [7]. Docker Compose orchestrates three services: the React frontend, the FastAPI backend, and the MongoDB database.

Containerization provides a consistent execution environment across different machines and simplifies project deployment. Developers can start the complete application using a single Docker Compose command without installing the individual software components manually.

9 Evaluation

This chapter evaluates the implemented prototype by demonstrating its main functionality. The evaluation focuses on the complete workflow of creating, validating, storing, and executing comparison formulas. Rather than measuring system performance, the objective is to verify that the implemented framework satisfies the functional requirements presented in Chapter 3.

9.1 Functional Evaluation

The developed prototype successfully implements all major features defined for the project. Users can create customizable comparison formulas using the proposed DSL, validate formulas before storage, execute stored formulas using custom input data, and receive explainable comparison results.

The framework also supports formula versioning, execution history, predefined comparison function management, and automatic synchronization of newly implemented comparison functions. Together, these features provide a flexible environment for defining and executing customizable comparison logic.

9.2 Example Comparison

To demonstrate the complete workflow, the Movie Metadata Consistency Check formula introduced in Chapter 6 was executed using the sample input presented in Chapter 7.

The execution engine evaluated the stored AST and invoked the required predefined comparison functions. The normalized movie titles matched, the publication years were identical, and the duration difference was within the allowed tolerance of twenty minutes.

Consequently, the framework classified the two records as **SIMILAR**. Besides the comparison label, the system generated a detailed explanation describing how each predefined comparison function contributed to the final result together with an overall confidence score.

Figure 5 shows the output produced by the execution engine. The interface presents the overall comparison result together with a detailed explanation for every comparison function and the calculated confidence score. This explainable output allows users to understand why the two movie records were classified as similar.

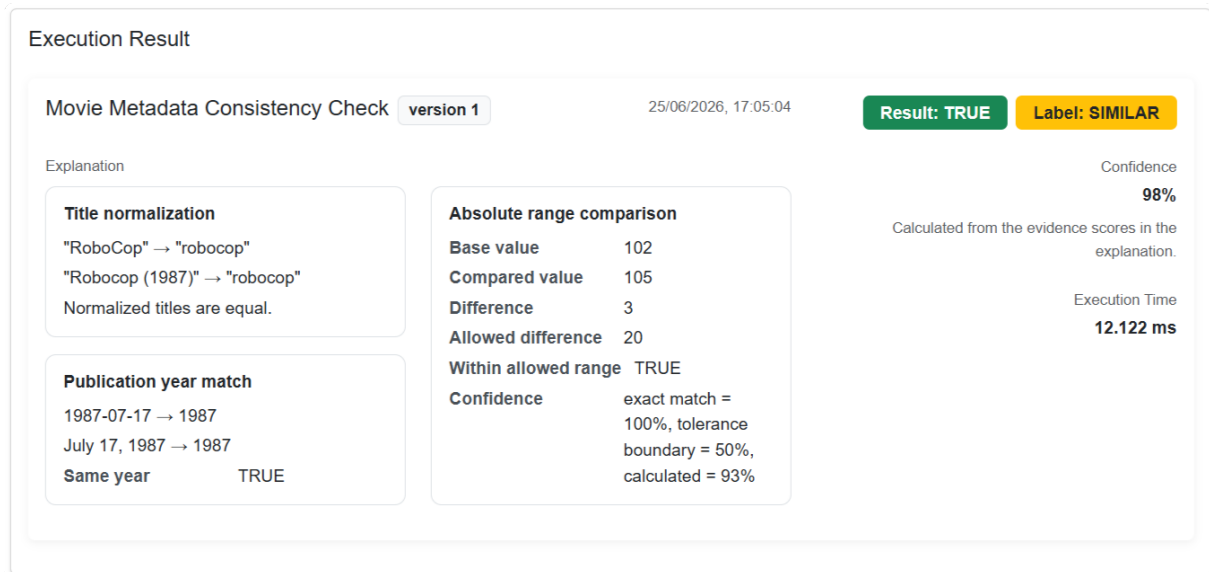


Figure 5: Execution result of the example comparison formula.

9.3 Discussion

The evaluation demonstrates that the proposed framework successfully supports customizable fact comparison without requiring users to modify the application source code. The combination of a domain-specific language, multi-stage validation, predefined comparison functions, and an AST-based execution engine enables flexible and reusable comparison logic while maintaining execution safety.

The generated explanations and confidence score improve transparency by allowing users to understand how comparison decisions are produced. Furthermore, the modular architecture allows new predefined comparison functions to be added without changing the language or the execution engine, making the framework easier to maintain and extend.

Table 1 summarizes the implementation status of the functional requirements presented in Chapter 3.

Table 1: Evaluation of the implemented features.

Requirement	Status
Formula creation	Implemented
Formula validation	Implemented
Formula versioning	Implemented
Formula execution	Implemented
Explainable results	Implemented
Function synchronization	Implemented
Docker deployment	Implemented

10 Challenges, Limitations and Future Work

This chapter discusses the main challenges encountered during the implementation of the project, the current limitations of the developed prototype, and possible directions for future improvements.

10.1 Challenges

Several challenges were encountered during the development of the comparison framework. One of the main challenges was designing a domain-specific language that is expressive enough to support different comparison scenarios while remaining simple and safe to use. Achieving this balance required careful design of both the language syntax and the validation process.

Another challenge was implementing the multi-stage validation pipeline. The system needed to distinguish between syntax errors, semantic errors, structural issues, and safety violations while providing meaningful feedback to users. Developing an AST-based execution engine that safely evaluates formulas without executing arbitrary Python code was another important implementation challenge.

Maintaining consistency between predefined comparison functions and the corresponding meta-data stored in the database also required additional consideration. This challenge was addressed by implementing an automatic synchronization mechanism that registers newly added predefined comparison functions during application startup.

10.2 Limitations

Although the implemented prototype satisfies the project objectives, several limitations remain.

Currently, users can only create formulas by combining predefined comparison functions. Extending the comparison capabilities still requires developers to implement additional predefined functions in Python before they become available within the DSL.

The framework has been evaluated using representative comparison scenarios but has not been tested on large-scale datasets. Consequently, the scalability and performance of the execution engine have not yet been evaluated in detail.

In addition, the prototype focuses primarily on customizable comparison logic and therefore does not implement optional features such as user authentication, authorization, role management, or collaborative editing of comparison formulas.

10.3 Future Work

The implemented framework provides a solid foundation for several engineering and research directions.

One promising research direction is the integration of AI-assisted formula creation. Instead of manually constructing comparison formulas, users could describe the desired comparison logic using natural language. A large language model (LLM) could then generate an initial DSL formula, which would subsequently pass through the existing parsing, validation, and execution pipeline. This approach could significantly simplify the creation of complex comparison formulas while preserving the safety guarantees provided by the framework.

Another potential extension is the intelligent recommendation and optimization of comparison formulas. By analysing previous execution results, validation feedback, and user interactions, the system could suggest improvements to existing formulas or recommend more suitable predefined comparison functions for a particular comparison task. Such recommendations could help users create more accurate and maintainable comparison logic.

Future work could also investigate graphical formula editors that allow users to construct comparison logic visually while automatically generating the corresponding DSL expressions. This would improve usability for domain experts with limited technical knowledge without changing the underlying execution model.

From a system engineering perspective, the execution engine could be optimized to support large-scale fact comparison involving thousands of formulas and datasets. Future evaluations should therefore investigate scalability, execution performance, and usability using real-world datasets from multiple application domains.

Finally, the proposed framework could be integrated into a complete fact-checking platform where customizable comparison formulas operate together with automated data acquisition, conflict detection, and result visualization. Such an integration would allow the framework to be evaluated as part of an end-to-end fact-checking workflow and provide further insight into its practical applicability.

11 Conclusion

This project presented a domain-specific language (DSL) for defining customizable fact comparison logic within the FactCheck framework. The proposed approach enables users to create reusable comparison formulas by combining predefined comparison functions with logical, arithmetic, comparison, and conditional operators while ensuring safe execution through Abstract Syntax Tree (AST)-based parsing and multi-stage validation.

The implemented prototype successfully supports the complete lifecycle of a comparison formula, including creation, validation, storage, execution, version management, and explainable result generation. The evaluation demonstrated that the developed framework satisfies the defined functional requirements and provides a flexible, maintainable, and extensible solution for customizable fact comparison.

Overall, the project demonstrates that combining a domain-specific language with controlled validation and execution provides a practical approach for implementing customizable comparison logic while maintaining safety, transparency, and maintainability. The developed framework establishes a solid foundation that can be extended with additional comparison functions and integrated into larger fact-checking systems in future work.

References

- [1] University of Vienna, “FactCheck – customizable comparison function logic,” 2026, project description for Practical Course 2.
- [2] Python Software Foundation, “ast — Abstract Syntax Trees,” 2025, accessed June 2026. [Online]. Available: <https://docs.python.org/3/library/ast.html>
- [3] M. Fowler, *Domain-Specific Languages*. Addison-Wesley, 2010.
- [4] Meta Platforms, Inc., “React documentation,” 2025, accessed June 2026. [Online]. Available: <https://react.dev/>
- [5] S. Ramírez, “FastAPI documentation,” 2025, accessed June 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [6] MongoDB Inc., “MongoDB documentation,” 2025, accessed June 2026. [Online]. Available: <https://www.mongodb.com/docs/>
- [7] Docker Inc., “Docker documentation,” 2025, accessed June 2026. [Online]. Available: <https://docs.docker.com/>
- [8] Schema.org Community Group, “Schema.org,” 2025, accessed June 2026. [Online]. Available: <https://schema.org/>
- [9] Python Software Foundation, “Python 3 documentation,” 2025, accessed June 2026. [Online]. Available: <https://docs.python.org/3/>